Reviewer Pack — Fourier Coefficient Flatness Implies Worst-Case Hardness for…

Entry 8f9c6a311a28 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 11:59:26 UTC

Fourier Coefficient Flatness Implies Worst-Case Hardness for Boolean Functions

Entry ID: `8f9c6a311a28` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-08 11:59:26 UTC

1. Conjecture

Field A (mathematical branch): FOURIER_ANALYSIS **Field B** (complexity object): CIRCUIT_COMPLEXITY

Statement:

> For a Boolean function f on n variables, if the average of the absolute values of its Fourier coefficients is less than 1/√n, then f is hard to compute in the worst case (i.e., requires exponential time).

Rationale (proposer's reasoning):

> Functions with small Fourier coefficients are 'flat' and have limited correlation with parity functions, suggesting they are hard to compute in the worst case. This connects average-case properties (small average Fourier coefficients) to worst-case hardness.

Taxonomy category: `AVG_TO_WORST_CASE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2003c22c09c54b3e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def circuit_complexity(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Function length must be a power of 2")

        def evaluate(inputs):
            x = inputs[0]
            for i in range(1, n):
                x ^= inputs[i] << i
            return f[x]

        max_depth = 0

        def dfs(depth, current_state):
            nonlocal max_depth
            if depth > max_depth:
                max_depth = depth
            if len(current_state) == n:
                evaluate(current_state)
                return
            for bit in [0, 1]:
                dfs(depth + 1, current_state + [bit])

        dfs(0, [])
        return max_depth

    def fourier_coefficient_magnitude(f):
        n = int(math.log2(len(f)))
        if len(f) != 2**n:
            raise ValueError("Function length must be a power of 2")
```

```
        sum_abs_values = sum(abs(x) for x in f)
        return sum_abs_values / len(f)

    n = random.choice([5, 10, 15, 20, 30, 40])
    f = generate_boolean_function(n)
    avg_coeff_magnitude = fourier_coefficient_magnitude(f)

    conjecture_holds = avg_coeff_magnitude < 1 / math.sqrt(n)
    counterexample = "" if conjecture_holds else "mapping_undefined"

    return {
        "metric_name": "average_fourier_coeff_magnitude",
        "metric_value": avg_coeff_magnitude,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    avg_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={avg_metric_value} std=0.0 support_fraction={support_fracti
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and higher resource allocation

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112250
2	propose	ollama_remote	qwen3:8b	0	0	112206
3	preregistration	ollama_remote	qwen3:8b	0	0	24138
4	novelty	ollama_remote	qwen3:8b	0	0	20728
5	novelty	ollama_remote	qwen3:8b	0	0	16118
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10962
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7870
8	judge	ollama_remote	qwen3:8b	0	0	53061

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 357333 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/8f9c6a311a28.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8f9c6a311a28.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/8f9c6a311a28.tar.gz` (if generated)